Project Proposal for Project 1

Team Number: 10

Team Members:

- Eddy Tian (42790253)
- Alexander Argatoff (87604641)
- Beichun Gu (15805476)
- Seiya Iwama (91173278)
- Du Pyo Park (51886159)
- Aakash Singh (63260442)
- Mandeep Singh (72040231)

Overview:

Project purpose or justification (UVP)

What is the purpose of this software? What problem does it solve? What is the unique value proposition? Why is your solution better than others?

The purpose of this software is to create a centralized, responsive web application that streamlines the TA allocation process at UBCO. It allows TA coordinators, instructors, and students to manage TA applications, course requirements, and assignments all in one place, replacing traditional, fragmented tools like email and spreadsheets.

What problem does it solve?

The current TA allocation process is inefficient, time-consuming, and prone to miscommunication due to manual coordination via email and disconnected tools. Our system addresses this by:

- Providing a structured platform for TA applications, instructor requests, and TA assignments
- Supporting automated schedule matching and CSV uploads for faster data handling
- Giving stakeholders a clear view of who is assigned to what, based on preferences, qualifications, and availability

What is the unique value proposition?

The TA-Portal offers a tailored solution specifically designed for UBCO's TA allocation process, combining visual scheduling, application tracking, and role-based access into one unified platform. Unlike generic scheduling tools or manual email-based coordination, our

system streamlines the entire workflow by allowing coordinators to manage TA assignments, instructors to submit course needs, and students to apply with preferences and availability, all through a secure, user-friendly interface. With features like automated conflict detection, CSV imports, email notifications, and real-time status updates, the TA-Portal ensures transparency, efficiency, and accuracy in TA management.

Why is your solution better than others?

- Built using industry-standard technologies (Spring Boot, React, MySQL, Docker), ensuring maintainability and performance
- Designed with a user-friendly interface using Tailwind and React for faster onboarding and better UX
- Enforces security best practices with JWT, HTTPS, and hashed passwords
- Easy to deploy and scale using Docker and REST APIs
- Focused on the specific workflows and needs of UBCO, ensuring relevance and usability out-of-the-box

High-level project description and boundaries

Describe your MVP in a few statements and identify the boundaries of the system.

The TA-Portal is a web-based platform designed to facilitate and streamline the process of assigning Teaching Assistants (TAs) to courses at UBCO. The system allows students to apply for TA positions, instructors to specify their course needs, and TA coordinators to manage applications and assignments efficiently. It includes role-based access, secure authentication, scheduling visualization, and automated email notifications.

System Boundaries and Included Features (MVP):

- User registration, login, and role management (student, instructor, coordinator)
- TA application submission with availability and course preferences
- Instructor input for TA requirements and grading hours
- Coordinator dashboard for managing assignments and viewing allocations
- Upload and parsing of CSV data (e.g., transcripts, course history)
- Weekly planner for schedule matching
- Email notifications for offers and updates
- MVP is done by July 4th (Mini-presentation Milestone)

Measurable project objectives and related success criteria (scope of project)

Make sure to use simple but precise statement of goals for the project that will be included when it the project is completed. Remember that goals must be clear and measurable and **SMART**. It should be clearly understood what success means to the project and how the success will be measured (as a high level, what is success?).

- **User Authentication:**
  Implement a secure login and registration system for three roles (student, instructor, coordinator) with password reset and account management features.
- **TA Application Process:**
  Enable students to submit TA applications, upload transcripts (CSV), specify preferences, and visualize their availability using a weekly planner.
- **Instructor Input and Course Needs:**
  Allow instructors to specify course requirements, grading hours, and view assigned TAs per course/lab.
- **TA Assignment Management:**
  Provide TA coordinators with tools to view applicants, match TAs to courses manually, and manage application statuses.
- **Communication and Notifications:**
  Implement automated email notifications for TA offer acceptance/denial and user inquiries.
- **Data Handling and Reporting:**
  Support upload of CSV files (past allocations, transcripts), and allow data export in CSV/PDF formats.

Success Criteria

- At least 90% of core functional requirements are implemented and tested successfully.
- All three user roles can log in, perform their key tasks, and interact through the system.
- All actions (apply, assign, update, notify) complete within 5 seconds.
- The system is deployed via Docker and documented clearly on GitHub.
- Users (coordinator, instructor, student) report that they can complete their tasks without outside tools (e.g., email, Excel).
- No major security flaws (passwords are hashed, no public data leaks).

Users, Usage Scenarios and High-Level Requirements

User Groups:

Provide a description of the primary users in the system and when their high-level goals are with the system (Hint: there is more than one group for most projects). Proto-personas will help to identify user groups and their wants/needs.

TA Coordinator/ Staff (Chad)

Admin staff who manages TA allocations each semester for UBCO

- Age: 40

Goals:

- Assign TAs courses and labs based on instructor needs and student applications
- Monitor past TA allocations and availability
- Upload course data via CSV

Frustrations:

- He is a very busy man. He has lots of aspirations. He wishes to advance his academic achievement in Mathematics while at the same time teaching multiple courses in MATH and STAT.

Behaviour:

- He is addicted to coffee and cannot go through a single day without coffee.

Needs:

- Dashboard to view applications and course needs
- Tools to match TA's with course requirements and schedules
- Send email notification for offers

Instructor (Scawt)

Course instructor with multiple classes needing TA support

- Age: 50. He has experience working in the industry and has been teaching Capstone or other related COSC courses for the past 15 years.

Goals:

- Submit TA requirements for each course/lab
- Specify prerequisites for each TA position
- View assigned TAs for current and past terms

Frustrations:

- He is a very busy man. He has lots of meetings with other professors very often.

Behaviour:

- He is also addicted to coffee and cannot go through a single day without coffee.

Needs:

- Simple interface to enter and update course needs
- View allocation status and history without needing technical knowledge

Student (Emma):

- Age: 21. She is a 3rd year undergraduate student.

Goals:

- to get valuable TA-experience.
- earn some money
- to get some connections with instructors. Connections can give chances for getting graduate study.

Frustrations:

- Emma is taking 4 courses this semester, and some of her courses are math-related courses, which gives her a lot of homework
- Emma is nervous about applying for the TA-role, because this is her first time trying to be a TA.
- Emma is worried if her TA-related assignments will interfere with her studying time

Behavior:

- Emma is a diligent student getting an average of A's for all her assignments and exams. She occasionally gets B's when some of the assignments and exams are unexpectedly difficult

Needs:

- Easy to use TA-application software with easy transcript upload and schedule planning
- Email notifications for offers and status changes
- Ability to Accept/Decline offers and cancel application if needed

Envisioned Usage

What can the user do with your software? If there are multiple user groups, explain it from each of their perspectives. These are what we called *user scenarios* back in COSC 341. Use subsections if needed to make things more clear. Make sure you tell a full story about how the user will use your software. An MVP is a minimal and viable, so don't go overboard with making things fancy (to claim you'll put in a ton of extra features and not deliver in the end), and don't focus solely on one part of your software so that the main purpose isn't achievable. Scope wisely. Don't forget about journey lines to describe the user scenarios.

Our TA Allocation and Management System is designed to support the process of allocating teaching assistants (TAs) to courses in a way that is efficient and manageable for all involved. The system focuses on facilitating TA applications for students and visualization for all users, with a clear and accessible desktop interface.

Scenario1:

Emma is a third-year undergraduate computer science student who wants to apply for a TA position for the upcoming term.

Sign Up & Profile Creation: Emma visits the website, signs up using her university email, and fills out her profile with her contact details, role (student), and transcript (uploaded from Workday).

Input Academic Info: She inputs the courses she has previously completed, selects her subject preferences (e.g., COSC, MATH), and uploads her weekly schedule using a CSV file from her student calendar.

Specify Preferences: Emma uses a weekly planner to select her available/preferred time sections and indicates that she is available for up to 8 hours a week.

Application Submission: Emma applies to TA for COSC and MATH.

Status Tracking: She logs in later to see her application status has changed to "under review".

Offer & Response: A month later, Emma receives an email saying she's been offered a TAship for COSC 121. She logs in and accepts the offer.

Planner View: She uses the weekly planner to visualize how the TA work fits into her schedule.

Scenario2:

Scawt is a computer science professor who is looking to input more details for the expectations he has of TA's for his 2 classes.

Sign up:

Scawt visits the website, signs up using his university email, and fills out his profile with his contact details and role (instructor)

Login & Course View: Scawt logs into the system and sees a dashboard listing his current and upcoming courses

Submit Needs: For each course, he inputs the number of hours needed for grading, and specifies that ideal candidates should have completed COSC 121 and COSC 211

View Allocation: A few weeks later, Scawt logs in again to see which TAs have been allocated to each course and lab

Minimal Interaction: He checks the allocations and logs out, knowing the coordinator will follow up if any changes are needed

Scenario3:

Ched is a TA coordinator for the Faculty of Science, he is responsible for reviewing and assigning TAs to courses based on availability and instructor needs

Login and Browse: Ched will login using the administrator account given by the system and sees a summary of all student applications and instructor requirements

Review Applications: He checks student-submitted availability and past course records, including transcripts

Match and Scheduling: Using the built-in weekly planner visualization, Ched matches students to courses where their schedules align

Modifying Schedules: He realises that the scheduling could be improved, so he edits some allocations

Send Offers: Once allocations are finalized, he sends out TA offers to students via student emails

Track Responses: Ched monitors acceptances and follows up with students who haven't responded.

Requirements:

In the requirements section, make sure to clearly define/describe the **functional** requirements (what the system will do), **non-functional** requirements (performance/development), **user requirements (what the users will be able to do with the system and **technical** requirements. These requirements will be used to develop the detailed uses in the design and form your feature list.

Functional Requirements:

- Describe the characteristics of the final deliverable in ordinary non-technical language
- Should be understandable to the customers
- Functional requirements are what you want the deliverable to do

Authentication and User Account Management:

- The system will be able to securely log in a user with email/username and password
- The system will be able to sign up a user with email/username and password.
- The system will be able to securely log out a user
- The system will allow users to change their email and password
- The system will allow users to delete their account
- The system will be able to update a password for a user

Course Scheduling:

- The system will store all courses in the upcoming semester to be viewed
- The system will be able to assign TA's to courses
- The system will be able to show instructors' needs
- The system will allow users to be removed
- The system will allow modify a user's profile information
- The system will be able to show information about past TA allocations
- The system will be able to read uploaded CSV data of past courses.
- The system will be able to show if a TA offer would fit with a student's schedule
- The system will be able to store and show how many hours of grading courses require
- The system will be able to update what courses TA's are required to have done to TA certain courses
- The system will be able to show every course a teacher is teaching
- The system will be able to show the allocations per course/lab
- The system will be able to let a student apply for a TA position
- The system will be able to let students specify number of hours he or she wants to work
- The system will be able to let students specify whether he or she wants to do it in person or remote
- The system will be able to read a CSV transcript from a student and show the results

- The system will be able to accept manually input courses that students have done
- The system will allow students to indicate preference for subjects (e.g., COSC, MATH)
- The system will be able to let students see application status.
- The system will automatically notify the student when the application is denied and/or accepted.
- The system will update the student's application status when the coordinator allocates a student to a course or denies their application.
- The system will send notifications by email with course information to TA's that are offered positions
- The system will be able let a student accept/deny new offers
- The system will be able to let a student, input the courses he will be doing in the upcoming semester (e.g., option to input schedule as CSV), when applying
- The system will be able to let students use a weekly planner application to show which hours they are available to TA
- The system will be able to let students see how their schedule fits visually on a weekly planner with detailed times
- The system will be able to let students cancel their application
- The system will be able to let a student fill a web-form with their questions or inquiries to the TA coordinator that automatically emails their message.
- The system will make the student profile show the hours they are available based on the courses they are taking.
- The system will make the course profile show the past/current TA allocations and more details of the course. (This requirement may change later depending on what we see in the CSV example we will get later)
- The system will allow the coordinator to create a questionnaire that is linked to the TA-application.
- The system will allow the student to fill out a questionnaire as he or she is doing the TA-application.

User-accessibility Feature

- The system will have a small tutorial to help first-time users (or users who just registered) to navigate through the website. This tutorial can be reviewed again.

Non-functional Requirements:

- Specify criteria that can be used to judge the final product or service that your project delivers
- List restrictions or constraints to be placed on the deliverable and how to build it; remember that this is intended to restrict the number of solutions that will meet a set of requirements.

Organizational Requirements:

- The system must meet all specifications and constraints defined in the Canvas project description.

- Codebase must be submitted via GitHub with appropriate documentation and usage instructions.

External requirements:
Legislative:
- The system must respect copyright laws by not including or copying proprietary content or software unless permitted.
- Any reused code or open-source components must follow their licensing conditions (e.g., MIT, GPL).
- The platform must comply with institutional data privacy rules, ensuring no unauthorized data sharing.

Security:
- The system must not disclose any personal information about the user apart from what they accepted to be public.

Product Requirements:
Reliability:
- When the system crashes, the data should still be the same. It is ACID compliant. Should either rollback or left in the state it was. Atomic, concurrency, isolated, durable

Portability:
- The system will use a design library like Tailwind to make the website responsive to the size of the screen.

Performance:
- The system cannot take more than 5 seconds to respond to a query.
- Weekly planner and application matching logic should complete within 5 seconds per query.

Space:
- The system must efficiently store data with indexing for faster queries and limited redundancy.

Security:
- The system must ensure that users' personal data is protected and only visible to authorized roles.
- All communication must be encrypted using HTTPS (TLS).
- Passwords must be stored using strong hashing algorithms.

Interoperability:
- System must be built using Docker for easy deployment.
- The system must accept CSV file uploads for student transcripts and course data.
- Reports and summaries must be exportable in PDF and CSV formats.

- External email services (e.g., SMTP) must be supported for sending notifications.

User Requirements:

- As a user, I will be able to login to the system using email/username and password
- As a user, I will be able to sign-up using name, contact info(phone number, email), role
- As a user, I will be able to update my email address, my name, password, my department, and other account related info.
- As a user, I will be able to logout of the system
- As a user, I will be able to recover password through email reset link
- As a user, I will be able to browse all courses in the current term
- As a user, I will be able to delete my account
- As a user, when I am first accessing this website, I will be able to navigate through the website easily.

TA coordinator and staff (admin):

- As a coordinator, I will be able to compare the TA's application and instructor's needs to allocate the TA to a course
- As a coordinator, I will be able to update the application status of a TA.
- As a coordinator, I can choose to notify the TAs when TA's application status gets updated. As a coordinator, I will be able to see instructors' needs
- As a coordinator, I will be able to view a TA's application.
- As a coordinator, I will be able to remove a user
- As a coordinator, I will be to modify a user's profile information
- As a coordinator, I will be able to see the information about past TA allocations of a TA.
- As a coordinator, I will be able to upload the CSV of past courses.
- As a coordinator, I will be able to visualize if a TA offer would fit with a student's schedule.
- As a coordinator, I will be able to view the information of past/current courses
- As a coordinator, I will be able to update/add the list of courses that require TA-allocation.
- As a coordinator, I will be able to accept/deny a student's application.
- As a coordinator, I will be able to create a questionnaire that the student must answer when filling out their application.

Instructors:

- As an instructor, I will be able to post/update what courses I require TA's to have done to this course.
- As an instructor, I will be able to include how many hours of grading I require.

- As an instructor, I will be able to see every course I am teaching
- As an instructor, I will be able to see the allocations per course/lab

Students:

- As a student, I will be able to apply for a TA position
- As a student, I will be able to specify number of hours I want to work
- As a student, I will be able to specify whether i want to do in person or remote
- As a student, I will be able to upload my transcript, which is from Workday
- As a student, I will be able to manually input the courses I've done
- As a student, I will be able to choose preference for subjects (e.g., COSC, MATH) in my application. I don't choose the exact subjects in my application.
- As a student, I will be able to see application status
  As a student, I will be able to view the details of the application already submitted.
- As a student, I will be notified by email with course information if I'm offered a position to TA a course
- As a student I will be able to accept/deny my offers
- As a student, when applying, I will be able to input the courses I will be doing in the upcoming semester (e.g., option to input schedule as CSV)
- As a student, I will be able to use a weekly planner application to show which hours I am available to TA
- As a student, I will be able to see how my schedule fits visually on a weekly planner with detailed times
- As a student, I will be able to cancel my application.
- As a student, I will be able to fill a web-form to send an email to the TA coordinator where I can raise any questions or enquiries.
- As a student, I will be able to fill out a questionnaire when doin g my Application.

Technical Requirements:

- These emerge from the functional requirements to answer the questions: – How will the problem be solved this time and will it be solved technologically and/or procedurally? – Specify how the system needs to be designed and implemented to provide required functionality and fulfill required operational characteristics.
- All the passwords must be hashed using strong algorithms before being stored in database (e.g., SHA256)
- We will be using JWTs for web authentication when using the system
- The system will allow filtering and keyword search for courses using dynamic UI elements like checkboxes and search bars.
- The weekly planner application used by the student/coordinator will be created by importing a third party library.
- The system will be built and run using Docker for easy portability for development and eventual deployment
- The System will have an email mailer for notifications and communication

- Use a RESTful API for CRUD operations and business logic
- A relational database Management System like MySQL will be used.
- A JDBC driver will be used to connect the java application to the database.
- All communication between client and server will use HTTPS to ensure encryption of data in transit

Tech Stack

Identify the "tech stack" you are using. This includes the technology the user is using to interact with your software (e.g., a web browser, an iPhone, any smartphone, etc.), the technology required to build the interface of your software, the technology required to handle the logic of your software (which may be part of the same framework as the technology for the interface), the technology required to handle any data storage, and the programming language(s) involved. You may also need to use an established API, in which case, say what that is. (Please don't attempt to build your API in this course as you will need years of development experience to do it right.) You can explain your choices in a paragraph, in a list of bullet points, or a table. Just make sure you identify the full tech stack. For each choice you make, provide a *short justification* based on the current trends in the industry. For example, don't choose an outdated technology because you learned it in a course. Also, *don't choose a technology because one of the team members knows it well.* You need to make choices that are good for the project and that meet the client's needs, otherwise, you will be asked to change those choices. Consider risk analysis.

Spring Boot: It is a popular and stable framework for building RESTful API's with a lot of documentation and resources. Plus it's in Java which we all know. It has easy build tools with Maven, and a number of packages for security, database access, RESTful endpoints, dependency injection, and object management. It limits the use of XML, so the application winds up being pure Java aside from a few dependency XML files. Since it's in Java, its power consumption and speed are good when compared to less efficient languages like Python or PHP.

React: It is a popular and stable framework. It has reusable components that can be used for less code duplication. It allows the developer to write HTML inside Typescript functions for an improved development experience. It has a huge collection of third-party libraries. It is the most popular framework used and has a huge amount of documentation, resources, and general help in building applications. It has enough speed for this application's purposes. Moreover, we will be using Vite to set up React quickly.

Tailwind CSS: Tailwind is a utility-first CSS framework that allows developers to style elements directly within HTML classes. It helps build responsive and consistent UIs quickly without writing custom CSS. Tailwind is lightweight, customizable, and popular for clean, maintainable styling.

MySQL: Our data is relational, we need a relational database, and MySQL is something we all know. Relational database management system known for its speed, reliability, and ease

of use. In our case, storing user profiles, course information, and TA assignments. It has a GPL license, and we are okay using that same license in our own software.

JDBC: It is the standard Java API for connecting and interacting with relational databases. It allows our application to execute SQL queries, retrieve results, and manage database connections directly from Java code. JDBC is fast and gives us fine-grained control over how we handle database operations, which is ideal for applications where performance and efficiency matter.

Docker: We will use Docker to containerize both backend and frontend components of our application, along with MySQL database. Managing with docker compose, we will ensure that applications can be set up and run consistently on any deployment environment.

Maven: It is a widely-used and stable build automation tool specifically designed for Java projects. It simplifies dependency management through its pom.xml file, allowing us to easily include libraries like Spring Boot and SQL drivers without manually downloading JAR files. Maven also streamlines the project structure, packaging, testing, and deployment processes with standard conventions. It integrates well with IDEs like IntelliJ and Eclipse, making development smoother.

High-level risks

Describe and analyze any risks identified or associated with the project.

- Failure to complete required features
    - Some key system functions may not be finished due to complexity or unclear requirements.
    - *Mitigation*: Prioritize essential features, clarify scope with the client early, and work in iterations.
- Time management issues
    - Project may fall behind schedule due to poor planning or competing academic deadlines.
    - *Mitigation*: Use a detailed project timeline with clear milestones and weekly progress check-ins.
- Miscommunication within the team
    - Misunderstandings can lead to incorrect implementations or duplicated work.

    - *Mitigation*: Hold regular meetings, document decisions, and use clear communication tools (e.g., Discord).

- Changing project requirements

    - The client may revise expectations mid-project, causing rework.
    - *Mitigation*: Design the system flexibly, and document all changes and their impacts before proceeding.

- Team member drops out or needs to go on bereavement leave.

    - Loss of a contributor could disrupt development and delay deliverables.
    - *Mitigation*: Share responsibilities, document all work, and ensure code is version-controlled for easy transition.

Assumptions and constraints

What assumptions is the project team making and what are the constraints for the project?

Assumptions

- All team members have a basic level of knowledge with the selected tech stack (e.g., web development, databases).
- An instructor is available to provide project goals, feedback, and approve deliverables. The instructor will relay any messages from the client.
- The application will be primarily used on desktop devices by instructors, students, and staff.
- The client's requirements will remain mostly stable once the initial scope is finalized.
- Team members will have access to necessary tools (e.g., GitHub, development environments, deployment platform).

Constraints

- **Timeframe**: The project must be completed within the academic term, with fixed deadlines for deliverables and presentations.
- **Limited availability**: Some team members have other courses, jobs, or responsibilities that restrict the number of hours they can contribute daily.
- **Technology scope**: Must use open-source or free tools only, and avoid overly complex integrations that require advanced infrastructure.
- **Client availability**: Client feedback may be delayed during holidays or busy periods, impacting iterations.

- **Academic policies**: Must follow university rules on academic integrity, group collaboration, and code originality.

Summary milestone schedule

Identify the major milestones in your solution and align them to the course timeline. In particular, what will you have ready to present and/or submit for the following deadlines? *List the anticipated features you will have for each milestone*, and we will help you scope things out in advance and along the way. Use the table below and just fill in the appropriate text to describe what you expect to submit for each deliverable. Use the placeholder text in there to guide you on the expected length of the deliverable descriptions. You may also use bullet points to clearly identify the features associated with each milestone (which means your table will be lengthier, but that's okay). The dates are correct for the milestones.

Milestone	Deliverable
May 27th	Project Plan Submission
June 3rd	Design Submission: Same type of description here. Aim to have a design of the project and the system architecture planned out. Use cases need to be fully developed. The general user interface design needs to be implemented by this point (mock-ups). This includes having a consistent layout, color scheme, text fonts, etc., and showing how the user will interact with the system should be demonstrated. ● System Architecture Design ● Use Case Models (Dup, Aakash) ● Database Design (Dup, Alex) ● Data Flow Diagram (Level 0 and Level 1) (Seiya, Allen) ● User Interface (UI) Design (Mandeep, Eddy)
June 6th	A short video presentation describing the design for the project. This will be reviewed and the team will receive feedback.
June 13th	Mini-Presentations: A short description of the parts of the envisioned usage you plan to deliver for this milestone. Should not require additional explanation beyond what was already in your envisioned usage. This description should only be a few lines of text long. This will be presented in the weekly team meeting. Remember that features also need to be tested. ● Login and Logout and register tested and completed (**Edited:** Rough draft of authentication) ● Course viewing and filtering and creation tested and completed (TA Coordinators) ● Student Profile (TA's perspective) tested and completed
June 20th	● All of the Instructor's input features related to TA-allocation (such as updating what courses the instructors need TAs for) tested and completed. ● TA Applications

Milestone	Deliverable
June 27th	● Instructor CSV-upload feature in frontend and backend tested and completed (doing this depends on whether we get a CSV example by this date or not) ● TA-coordinator's scheduling feature tested and completed.
July 4th	MVP Mini-Presentations: A short description of the parts of the envisioned usage you plan to deliver for this milestone. Should not require additional explanation beyond what was already in your envisioned usage. This description should only be a few lines of text long. Feature set #1 will be completed by this milestone. Remember that features also need to be tested. *Clients will be invited to presentations.* ● All basic functionality is done for MVP. ● TA's visual allocation on assignments feature completed and tested ● Instructor's visual allocation features tested and completed and tested
July 11th	Peer testing and feedback: Aim to have an additional features implemented and tested by team member. As the software gets bigger, you will need to be more careful about planning your time for code reviews, integration, and regression testing. ● Authentication tested and completed ● TA-coordinator admin features/privileges tested and completed ● Exam feature may implemented.
July 18	● TA-profile edit features tested and completed ● Instructor's profile edit features tested and completed ● First-time user accessibility assisting features tested and completed
July 25th	Test-O-Rama: Full scale system and user testing with everyone ● Any other miscellaneous features tested and completed.
August 8th	Final project submission and group presentations - Feature set #2: Details to follow

Teamwork Planning and Anticipated Hurdles

Based on the teamwork icebreaker survey, talk about the different types of work involved in a software development project. Start thinking about what you are good at as a way to get to know your teammates better. At the same time, know your limits so you can identify which areas you need to learn more about. These will be different for everyone. But in the end, you all have strengths and you all have areas where you can improve. Think about what those are, and think about how you can contribute to the team project. Nobody is expected to know everything, and you will be expected to learn (just some things, not everything). Use the table below to help line up everyone's strengths and areas of improvement together. The table should give the reader some context and explanation about the values in your table.

For **experience** provide a *description of a previous projec*t that would be similar to the technical difficulty of this project's proposal. None, if nothing For **good At**, list of skills relevant to the project that you think you are good at and can contribute to the project. These could be soft skills, such as communication, planning, project management, and presentation. Consider different aspects: design, coding, testing, and documentation. It is not just about the code. You can be good at multiple things. List them all! It doesn't mean you have to do it all. Don't ever leave this blank! Everyone is good at something!

Category	Aakash	Alex	Allen	Dup
Experience	My 310 project was similar to the one we're doing now. The project was a web app that used vaadin, HTML/CSS, and Javascript for the front end and MySQL and JDBC for the backend. I had done both backend and frontend for this project. Additionally, i have worked on a couple of personal projects which utilized node.js, react, and java/python.	I have experience with full stack applications, including a full stack NextJS project done for 310, and a music organization tool I've co-developed called Jampal for mobile devices using Flutter with an Express backend. I have experience with Postgres, SQLite and MySQL, and a number of languages. I have good experience with Docker and general backend implementations, and some React experience for frontend.	I've worked on several web applications using Flask, Express, and MySQL. One of my projects was a collaborative course management tool built using Flask for the backend and HTML/CSS/JavaScript for the frontend. It allowed students to track assignments, grades, and announcements, with MySQL handling relational data storage. I was responsible for both backend API development and frontend integration.	A previous project similar to this project is the project I did in my 310 class. The 310 project was also a web app that used React but Postgres and node for backend. The size and scale of the project in 310 is similar to that of this class. In that project, I did both backend and frontend, except for the database design.
Good At	I'm comfortable working with web technologies like HTML, CSS, and JavaScript, and I've spent a lot of time building and troubleshooting front-end components. I've also worked with tools like Git and VS Code for collaborative development. I consider myself adaptable when learning new	I would consider myself good at reading documentation to try and write good, maintainable code. I'm competent at Docker, and have a good understanding of RESTful api's and building them. I have used a variety of databases, and have a strong understanding of database design and implementations. I consider myself to be a fast learner, and a good	I'm comfortable working across the full stack, especially with backend frameworks like Flask and Express. I'm familiar with RESTful API design, MySQL integration, and handling user authentication. I pick up new technologies quickly and enjoy debugging and problem solving. I've used Git consistently	Some skills that I have that would help this project is my experience with React. Some soft skills I have are video editing, communication, and presentation. I may have more experience than others with design (CSS) because I played with Codepen several years ago. Therefore, I am relatively prepared

Category	Aakash	Alex	Allen	Dup
	frameworks or libraries, and I enjoy working in teams where open communication and clear task delegation are encouraged. While I'm not an expert in every area, I always put in the effort to understand what needs to be done and ask questions when needed.	communicator with various teams I've been in, and overall can work well with a variety of different people.	in team projects and prioritize writing clean, readable code. I communicate well with teammates and make sure tasks are clearly divided and tracked.	for any tasks that require moderate CSS modification, though I can't say I'm very confident at it. I would argue that I am an okay communicator, because I make sure to resolve conflicts by communicating first and I try very hard to get my ideas and feelings across.
Expect to learn	I expect to learn Spring Boot and Docker. I would also like to get more familiar with React.	I expect to learn Spring Boot, and how to build a good REST api using this framework. I also plan on getting more familiar with React and also Docker and microservices.	I expect to learn Spring Boot and Docker. Also micro services.	I expect to learn Spring Boot and Docker. Anything Dev-op related tasks is something I always expect to learn more of. I am a little more confident in Spring Boot, because I tried playing with it for a short time. I don't imagine it will be difficult, but I expect to learn incorporating Tailwind into a React Project.

Category	Eddy	Mandeep	Seiya
Experience	I have significant experience with web development, especially frontend through previous courses such as COSC 360 and 310. In COSC 310, I worked on a Canvas clone where we used Flask and HTML/CSS/Javascript, and in COSC 360 we used the XAMPP stack to create a Reddit-style social media	I have hands-on experience with Java, HTML, CSS, PHP, JavaScript, Python, Bootstrap, MySQL, and the LAMP stack. In COSC 360, our team of two built a full-stack web project using LAMP, where I was responsible for both frontend and backend development. In COSC 310, I worked on a Java-based project using	In COSC 310, my team developed a Discord-style chat application. We used React, Node.js/Express, MongoDB, and MQTT. My contributions included designing the friend-and-group data model, adding JWT authentication, enabling real-time messaging, and creating React components.

Category	Eddy	Mandeep	Seiya
	app. In both projects, I focused on frontend development but I also helped with backend issues when needed. I also have experience with MySQL through COSC 304.	NetBeans and MySQL, where our team developed the entire system, including frontend, backend, and testing components. These projects helped strengthen my full-stack and collaborative development skills.	In COSC 360 I co-developed an e-commerce site on a LAMP stack (PHP, MySQL, Bootstrap, JavaScript). I implemented secure login and registration with prepared statements, built the shopping-cart and checkout flows, added order-history and an admin dashboard, and deployed to the UBC server. These projects are comparable in scope to our current work and have given me practice in coordinating front-end and back-end work.
Good At	I'm most comfortable working on aspects of web development that are related to the frontend as I have a lot of experience working with HTML, CSS and Javascript. I am also very comfortable with handling any relational database-related issues. On the backend side, I feel confident working with PHP.	Good at database and frontend technologies such as HTML, CSS, and server-side scripting with PHP. Also proficient in working with MySQL for designing, querying, and managing relational databases to support dynamic web applications.	I'm comfortable with front-end development, particularly with html/css, and JavaScript. I've built responsive interfaces using Bootstrap and implemented dynamic features with JavaScript. On the back end, I have experience using PHP to build secure, data-driven functionality, and working with MySQL to design and query relational databases.
Expect to learn	As we are using modern frontend frameworks such as React and Tailwind that were only lightly touched on in my previous courses, I expect to learn how to comfortably use these tools. I am also expectring to become more familiar with Docker and learn Spring Boot.	I am expecting to learn and gain practical experience with modern tools and frameworks such as React for building interactive user interfaces, Tailwind CSS for efficient styling, Docker for containerization and deployment, and Spring Boot for developing robust Java-based backend services.	I expect to learn how to develop robust backend services using Spring Boot, including RESTful API design and secure authentication. I also hope to deepen my understanding of CI/CD workflows and deployment pipelines. While I've primarily styled projects with Bootstrap, I'd like to learn Tailwind CSS to better apply utility-first design principles.

Use this opportunity to discuss with your team who **may** do what in the project. Make use of everyone's skill set and discuss each person's role and responsibilities by considering how everyone will contribute. Remember to identify project work (some examples are listed below at the top of the table) and course deliverables (the bottom half of the table). You might want to change the rows depending on what suits your project and team. Understand that no one person will own a single task. Recall that this is just an incomplete

example*. Please explain how things are assigned in the caption below the table, or put the explanation into a separate paragraph so the reader understands why things are done this way and how to interpret your table. Please note that this is just an example table and that you will need to complete this in more detail.*

Category of Work/Features	Aakash	Alex	Allen Gu	Dup	Eddy	Mandeep	Seiya
Project Management Kanban Board Maintenance:	✔	✔	✔	✔	✔	✔	✔
System Architecture Design		✔					
User Interface Design					✔	✔	
CSS Development				✔	✔	✔	✔
DFD			✔				✔
Use Case	✔			✔			
Database Design		✔					
User Accessibility Documentation				✔	✔	✔	✔
Database/microservice setup		✔					
Frontend setup				✔		✔	
Presentation Preparation (Powerpoint)					✔		✔
Design Video Creation and Editing	✔		✔				
Weekly Team Log	✔	✔	✔	✔	✔	✔	✔

Category of Work/Features	Aakash	Alex	Allen Gu	Dup	Eddy	Mandeep	Seiya
MVP mini presentation	✔	✔	✔	✔	✔	✔	✔
Final Team Report	✔	✔	✔	✔	✔	✔	✔

Explanation of Table:

The rows for features are deleted. The reason is that everyone will work on the large features that need to be done. It doesn't make sense for one person to do nothing because the current milestone doesn't require that feature to be implemented yet. Therefore, this table only shows non-functional requirements or technical requirements or milestones of the team.

The final team report, project management and the MVP mini presentation have been checked for all columns because everyone is expected to contribute to those three things. Everyone should be updating the kanban board, have something to write in the final team report, and be participating in the MVP presentation. Moreover, the weekly team log will be rotated. Every week, there will be a different person doing the weekly team log.

The rows such as DFDs and Database design have been added because there has to be someone in charge of those charts as the project moves on. As the project moves on, these things can change.